Universidad Central

ECN-24 Desarrollo web en entorno servidor

Cancha Clara

Propuesta del Avance 1 para un sistema de reservación de canchas deportivas

Estudiante
Nicole Jimenez

Docente
Gilder López

Fecha de entrega
6 de noviembre de 2026

Repositorio
https://github.com/nidejimenezs/cancha-clara-ecn24

Problema contexto y alcance

Se propone una solución para un centro deportivo ficticio de Costa Rica que alquila canchas de fútbol cinco y baloncesto. Cuando las reservas se anotan en mensajes o cuadernos separados, recepción debe revisar varias fuentes para confirmar un horario. Esto facilita duplicar turnos y perder la relación entre una reserva y sus abonos.

Cancha Clara busca reunir la información de canchas, clientes, reservas y pagos. El resultado esperado es que recepción pueda identificar un turno disponible, registrar quién lo solicita y consultar cuánto falta por pagar. Las tarifas y reglas de este documento son supuestos del caso académico, no información recopilada de una empresa real.

Usuarios

Usuario	Necesidad
Recepcionista	Consultar horarios, registrar clientes y reservas, y anotar abonos sin mezclar pagos.
Administrador	Mantener canchas y tarifas, marcar mantenimiento y revisar reservas y saldos.
Cliente	Solicitar una cancha y recibir confirmación del horario y del monto. En esta versión lo atiende recepción.

Alcance del producto propuesto

La aplicación final administrará cuatro entidades relacionadas. Canchas y clientes tendrán CRUD completo; las reservas tendrán consulta, creación y cancelación; los pagos se registrarán como abonos. El flujo trabajará en colones enteros, con bloques de una hora dentro del mismo día y horario local de Costa Rica.

Alcance comprobable del Avance 1

Esta entrega incluye el modelo en TypeScript, ocho reglas demostradas en consola, cinco registros por entidad, pruebas, cuatro bocetos y un inicio Angular con tres componentes. Los datos viven en memoria y se reinician al ejecutar.

Quedan fuera de este avance la API, MongoDB, las rutas, los formularios interactivos y la autenticación. Del producto propuesto quedan fuera la conexión bancaria, la facturación electrónica, los torneos y la venta de implementos. Registrar un pago SINPE significará anotar su comprobante, no consultar una cuenta bancaria.

Requerimientos funcionales

Estos requerimientos describen el producto a construir por etapas. La prioridad indica qué es necesario para atender reservas correctamente; no significa que todas las pantallas estén implementadas en el Avance 1.

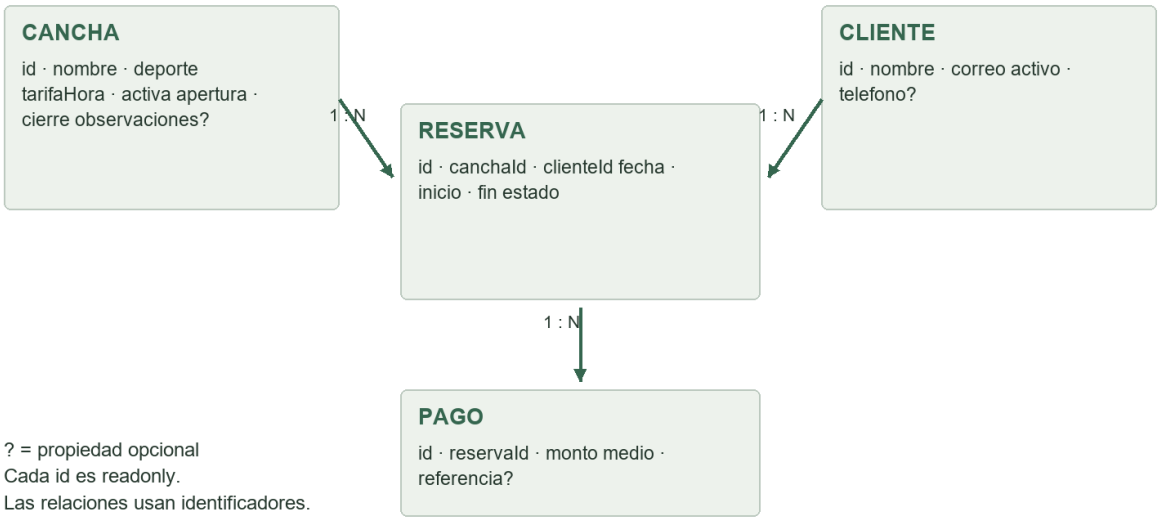
ID	Descripción verificable	Prioridad
RF-01	Crear, listar, consultar, editar y eliminar canchas. Impedir eliminar una cancha con reservas asociadas.	Alta
RF-02	Crear, listar, consultar, editar y eliminar clientes. Impedir eliminar un cliente con reservas asociadas.	Alta
RF-03	Filtrar las canchas por deporte y mostrar solo las activas al iniciar una reserva.	Alta
RF-04	Consultar la disponibilidad de una cancha por fecha y rango de horas.	Alta
RF-05	Crear una reserva para un cliente activo; rechazar cruces de horario, fechas inválidas y turnos fuera de atención.	Alta
RF-06	Calcular el total como tarifa por hora multiplicada por duración y mostrarlo antes de registrar el cobro.	Alta
RF-07	Registrar abonos en efectivo o SINPE; exigir referencia para SINPE, impedir duplicados y montos mayores al saldo.	Alta
RF-08	Listar reservas por fecha, cancha, cliente y estado, con acceso a su detalle y saldo.	Media
RF-09	Cancelar reservas sin pagos, liberando el horario; si hay abonos, derivar el caso a recepción para revisión.	Media
RF-10	Mostrar confirmación o un mensaje de validación concreto después de guardar un registro.	Media

Criterio de ejemplo para RF-05: si C01 tiene una reserva de 18:00 a 19:00, el sistema rechaza otra de 18:00 a 20:00 y acepta una de 19:00 a 20:00, siempre que las demás condiciones sean válidas. La cancelación y la eliminación con relaciones se implementarán en etapas posteriores.

Calidad y modelo de datos

ID	Requerimiento no funcional y verificación
RNF-01	Compilar el prototipo y el frontend con strict, sin errores y sin any en el código propio. Verificar con npm run build.
RNF-02	Mostrar el inicio sin desplazamiento horizontal a 390 y 1280 píxeles de ancho en navegadores Chromium actuales.
RNF-03	En la versión integrada, responder consultas de disponibilidad en menos de 2 segundos para 500 reservas en una prueba local de 20 consultas. Es una meta, no una medición de este avance.
RNF-04	Mantener credenciales fuera del repositorio y excluir node_modules, dist y .env. Revisar el contenido antes de publicar.

Figura 4 Entidades de Cancha Clara



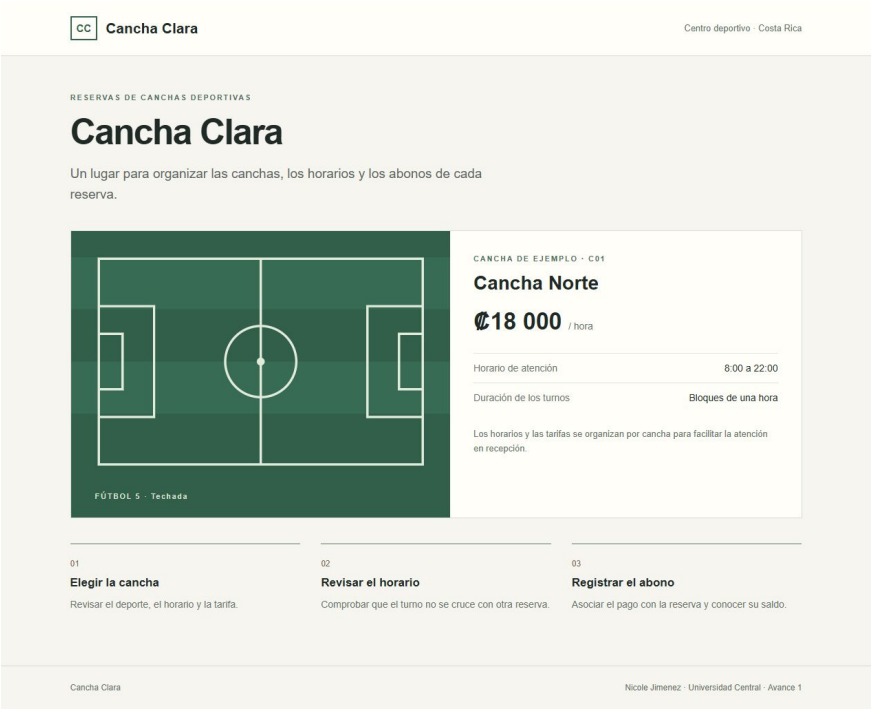
Modelo de datos. Una cancha tiene muchas reservas; un cliente puede realizar muchas reservas; una reserva recibe cero o más pagos.

Cada entidad tiene un id de solo lectura. Cancha incluye deporte, tarifaHora, estado activo y horas de atención; Cliente incluye nombre, correo y estado activo; Reserva contiene las referencias, fecha, horas y estado; Pago contiene referencia de reserva, monto y medio. observaciones, telefono y referencia son opcionales; SINPE exige esta última por regla de negocio.

Las relaciones se representan mediante identificadores. Se evita copiar dentro de cada reserva todos los datos del cliente o la cancha. El diagrama muestra contratos de datos, no colecciones de MongoDB ya implementadas.

Diseño y evidencia del avance

Los cuatro bocetos se incluyen en bocetos/ y se explican en la guía visual. El recorrido previsto es Inicio → Lista de canchas → Detalle y disponibilidad → Formulario de reserva. Guardar valida y vuelve al detalle; cancelar regresa sin crear el registro.



Captura real del inicio Angular ejecutándose. La ilustración de la cancha es un SVG elaborado para este proyecto; los datos se muestran mediante interpolación.

El componente Inicio contiene una Cancha tipada y los getters tarifaFormateada y horario. Encabezado, Inicio y PiePagina fueron generados con Angular CLI y se usan en app.html. El diseño se adapta a pantallas pequeñas y mantiene una jerarquía simple: proyecto, cancha de ejemplo y explicación del proceso.

El prototipo demostró dos reglas por entidad principal: canchas activas y horario; correo único y límite de cliente; disponibilidad y total; saldo y validación de abonos. Repositorio<T> agrega, busca, elimina y filtra. TarifaRegular y TarifaPromocional demuestran polimorfismo.

Verificación. Los dos programas TypeScript compilaron; el prototipo pasó seis pruebas automatizadas. Angular compiló y se abrió con ng serve. La guía técnica explica cómo ejecutar el proyecto y comprobar estos resultados.

Siguiente etapa. Implementar navegación, formularios y datos por HTTP. El modelo y las reglas deberán conservarse al incorporar persistencia y validaciones en el servidor.